Generative Adversarial Nets

Ph.D. Preliminary Exam Presentation

Shreyas Rajesh
Advisor : Prof. Vwani Roychowdhury

UCLA

November 30, 2023

Outline

1. Generative Models
   - Goal of Generative Models
   - Implementation of Generative models
2. Generative Adversarial Networks (GAN)
   - GAN: Problem Setup
   - GAN: Framework
   - GAN: Training
3. Results
   - Results
   - Drawbacks
4. Impact and Future Work
   - Impact
   - Drawbacks and Future Work

Generative Models : Goal

- **Goal**

Generative Models : Goal

- **Goal** : From samples learn the underlying distribution.

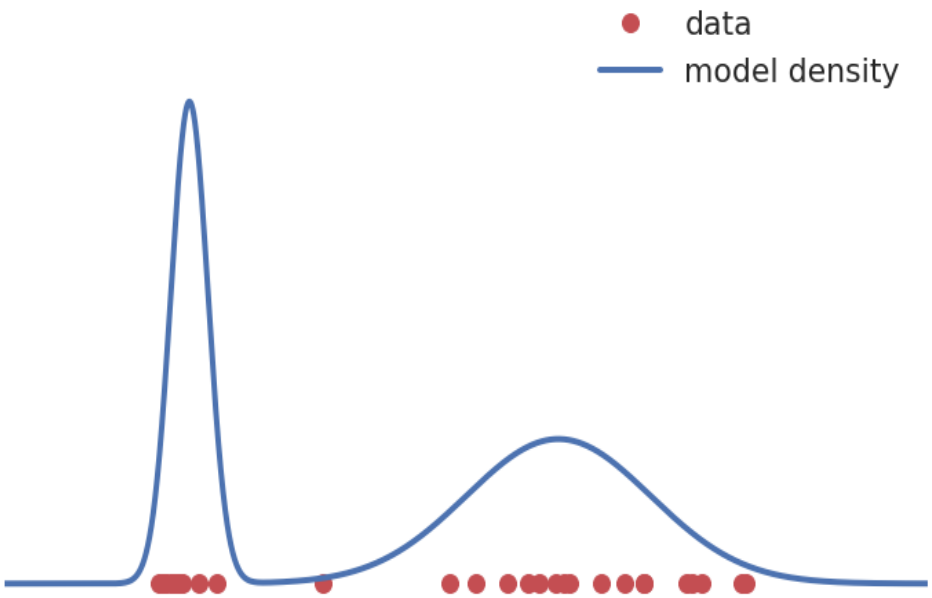


Image Credit: *Murphy 2023*

Generative Models : Goal

- **Goal** : From samples learn the underlying distribution.

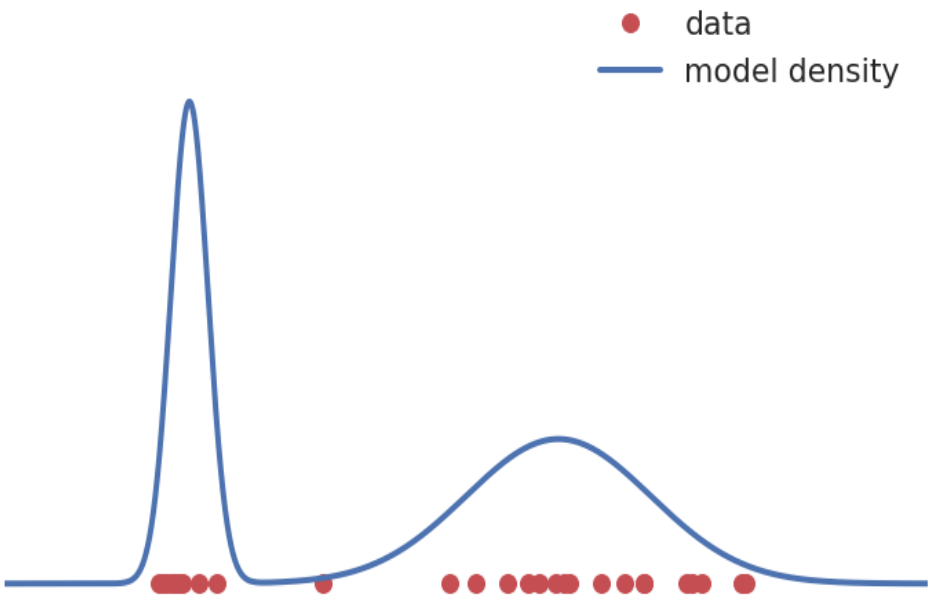


Image Credit: *Murphy 2023*

- Can we do this for more complicated distributions? Like Images, Text or both?

Deep Generative Models : Applications

Deep Generative Models : Applications

- **Image Generation**: Create realistic images, or modify existing images.

Deep Generative Models : Applications

- **Image Generation**: Create realistic images, or modify existing images.



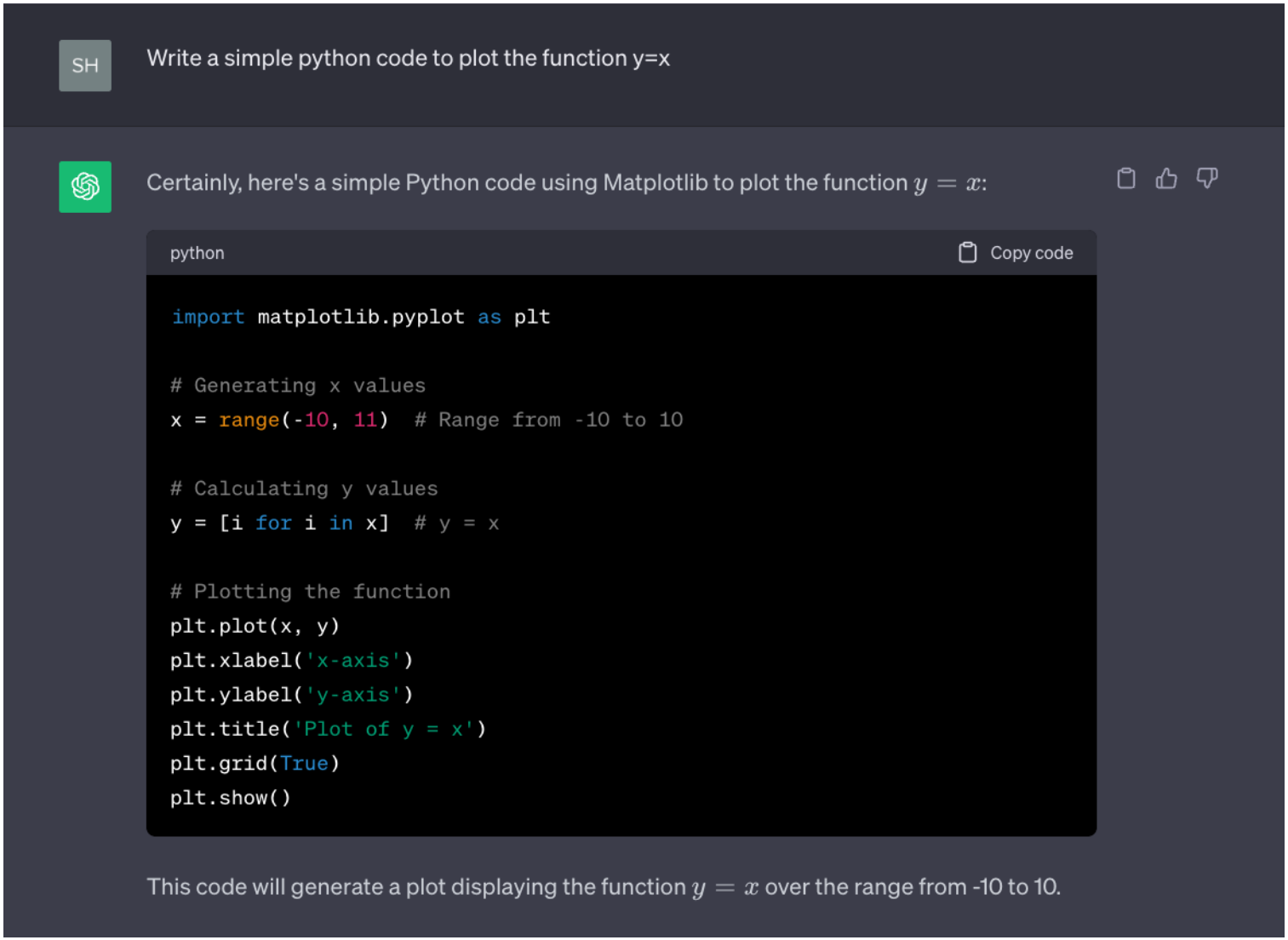
Deep Generative Models : Applications

Deep Generative Models : Applications

- **Text Generation**: Generate code, stories or summaries of documents.

Deep Generative Models : Applications

- **Text Generation**: Generate code, stories or summaries of documents.



Generative Models : Implementation

Generative Models : Implementation

- **Goal**: Learn a distribution $p_{model}(x)$ from samples $x_1, x_2, \ldots, x_N$.

Generative Models : Implementation

- **Goal**: Learn a distribution $p_{model}(x)$ from samples $x_1, x_2, \ldots, x_N$.
- One approach : **Latent Variable Models.**

Generative Models : Implementation

- **Goal**: Learn a distribution $p_{model}(x)$ from samples $x_1, x_2, \ldots, x_N$.
- One approach : **Latent Variable Models.**
- **Idea**: There exists a latent variable z such that

$$p(x) = \int p(x|z)p(z)dz$$

Generative Models : Implementation

- **Goal**: Learn a distribution $p_{model}(x)$ from samples $x_1, x_2, \ldots, x_N$.
- One approach : **Latent Variable Models.**
- **Idea**: There exists a latent variable z such that

$$p(x) = \int p(x|z)p(z)dz$$

- How do we compute this integral?

Generative Models : Tractable Integral

Generative Models : Tractable Integral

- Generative model:

$$z \sim \text{Categorical}_{\pi}(K)$$

$$x \sim N(\mu_k, \Sigma_k)$$

$$\mu_k \in \mathbb{R}^d, \Sigma_k \in \mathbb{R}^{d \times d}$$

Generative Models : Tractable Integral

- Generative model:

$$z \sim \text{Categorical}_{\pi}(K)$$

$$x \sim N(\mu_k, \Sigma_k)$$

$$\mu_k \in \mathbb{R}^d, \Sigma_k \in \mathbb{R}^{d \times d}$$

- $p(x) = \sum_{k=1}^{K} \pi_k N(x|\mu_k, \Sigma_k)$

Generative Models : Tractable Integral

- Generative model:

$$z \sim \text{Categorical}_{\pi}(K)$$

$$x \sim N(\mu_k, \Sigma_k)$$

$$\mu_k \in \mathbb{R}^d, \Sigma_k \in \mathbb{R}^{d \times d}$$

- $p(x) = \sum_{k=1}^{K} \pi_k N(x|\mu_k, \Sigma_k)$
- This is a **Mixture of Gaussians** model.

Generative Models : GMM

Generative Models : GMM

- **Goal**: Learn a distribution $p_{model}(x)$ from samples $x_1, x_2, \ldots, x_N$ assumed to be i.i.d.

Generative Models : GMM

- **Goal**: Learn a distribution $p_{model}(x)$ from samples $x_1, x_2, \ldots, x_N$ assumed to be i.i.d.
- Standard Approach : Maximize the likelihood of the data samples.

$$\max_{\theta} \prod_{i=1}^{N} p_{model}(x_i; \theta)$$

Generative Models : GMM

- **Goal**: Learn a distribution $p_{model}(x)$ from samples $x_1, x_2, \ldots, x_N$ assumed to be i.i.d.
- Standard Approach : Maximize the likelihood of the data samples.

$$\max_{\theta} \prod_{i=1}^{N} p_{model}(x_i; \theta)$$

- The parameters are estimated using the Expectation Maximization(EM) algorithm.

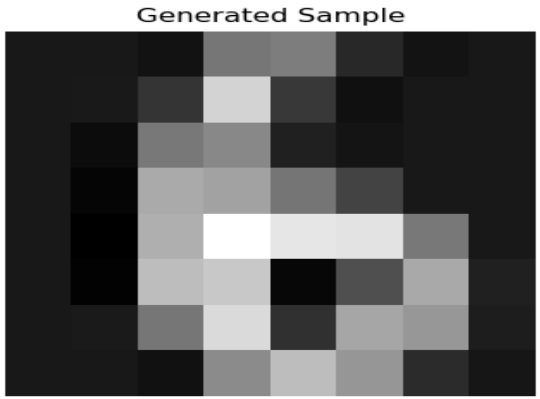
Generative Models : GMM results

Generative Models : GMM results

- How does the model perform specifically in high dimensional tasks?

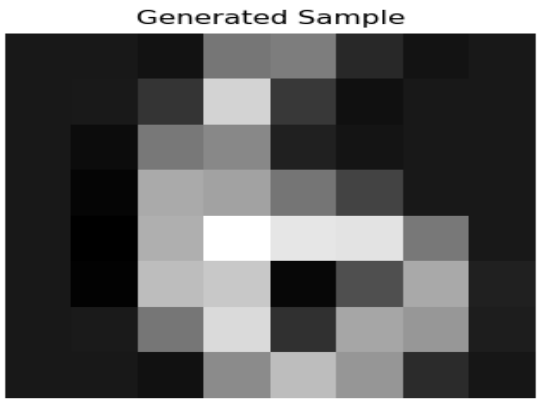
Generative Models : GMM results

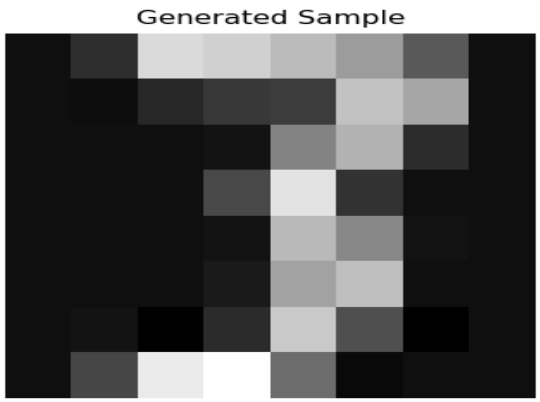
- How does the model perform specifically in high dimensional tasks?



Generative Models : GMM results

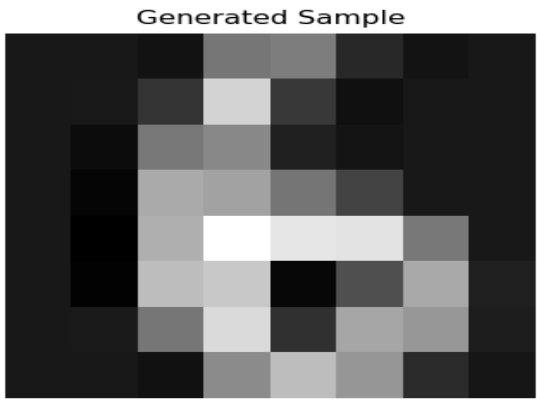
- How does the model perform specifically in high dimensional tasks?

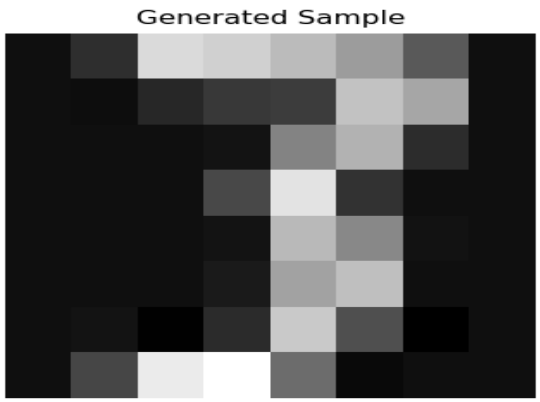


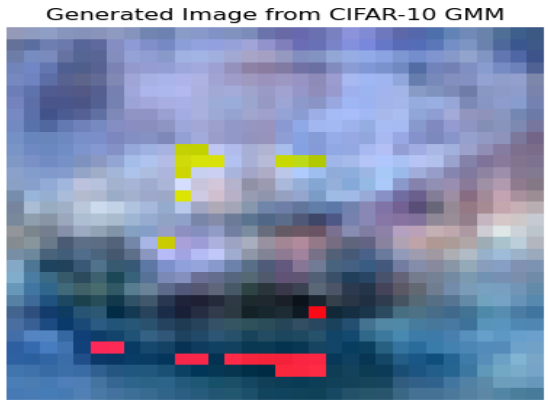


Generative Models : GMM results

- How does the model perform specifically in high dimensional tasks?

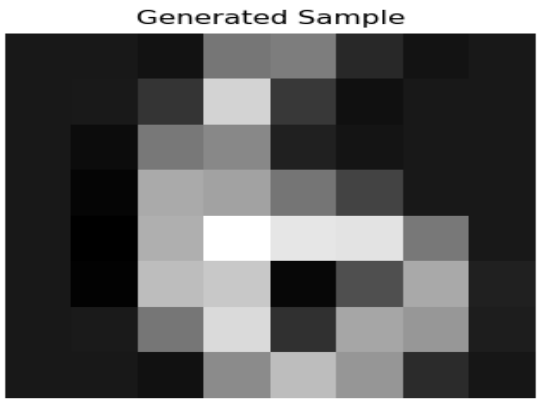


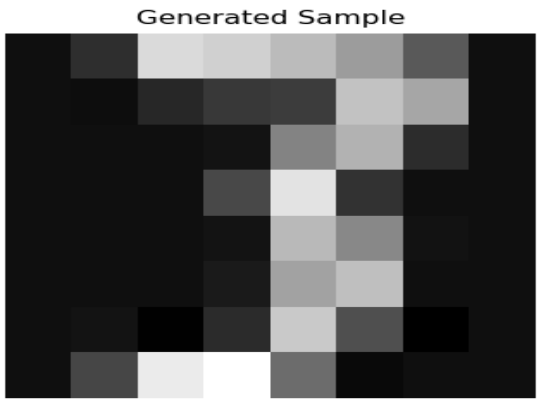


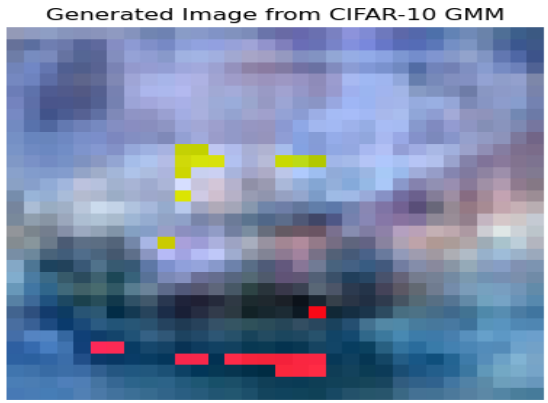


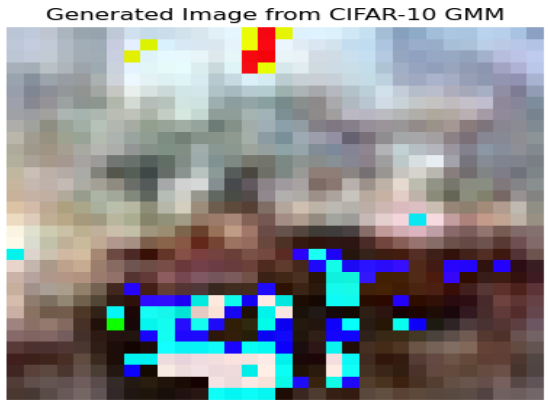
Generative Models : GMM results

- How does the model perform specifically in high dimensional tasks?









Generative Models : Intractable Integral

Generative Models : Intractable Integral

- Considering the next more likely case : Integral is intractable.

$$p(x) = \int p(x|z)p(z)dz$$

Generative Models : Intractable Integral

- Considering the next more likely case : Integral is intractable.

$$p(x) = \int p(x|z)p(z)dz$$

$$p(x) = E_{z\sim p(z)}[p(x|z)]$$

Generative Models : Intractable Integral

- Considering the next more likely case : Integral is intractable.

$$p(x) = \int p(x|z)p(z)dz$$

$$p(x) = E_{z\sim p(z)}[p(x|z)]$$

- Use Monte Carlo Sampling to estimate the integral.

$$p(x) \approx \frac{1}{N}\sum_{i=1}^{N} p(x|z_i)$$

Generative Models : Intractable Integral

Generative Models : Intractable Integral

- Sample z_i from $p(z)$ and compute $p(x|z_i)$.

$$p(x) \approx \frac{1}{N} \sum_{i=1}^{N} p(x|z_i)$$

Generative Models : Intractable Integral

- Sample z_i from $p(z)$ and compute $p(x|z_i)$.

$$p(x) \approx \frac{1}{N} \sum_{i=1}^{N} p(x|z_i)$$

- **Problem**: How do we sample from $p(z)$ such that we get good samples?

Generative Models : Intractable Integral

- Sample z_i from $p(z)$ and compute $p(x|z_i)$.

$$p(x) \approx \frac{1}{N} \sum_{i=1}^{N} p(x|z_i)$$

- **Problem**: How do we sample from $p(z)$ such that we get good samples?
- Due to the curse of dimensionality, sampling from $p(z)$ is difficult.

GAN setup

GAN setup

- **Goal:** We want to simulate a true data distribution $p_{true}(x)$.

GAN setup

- **Goal:** We want to simulate a true data distribution $p_{true}(x)$.
- **Problem:** Trying to maximize likelihood leads to an intractable integral.

GAN setup

- **Goal:** We want to simulate a true data distribution $p_{true}(x)$.
- **Problem:** Trying to maximize likelihood leads to an intractable integral.
- What if instead of specifying a likelihood we are only concerned with generating samples

GAN setup

- **Goal:** We want to simulate a true data distribution $p_{true}(x)$.
- **Problem:** Trying to maximize likelihood leads to an intractable integral.
- What if instead of specifying a likelihood we are only concerned with generating samples
- Implicit generative models

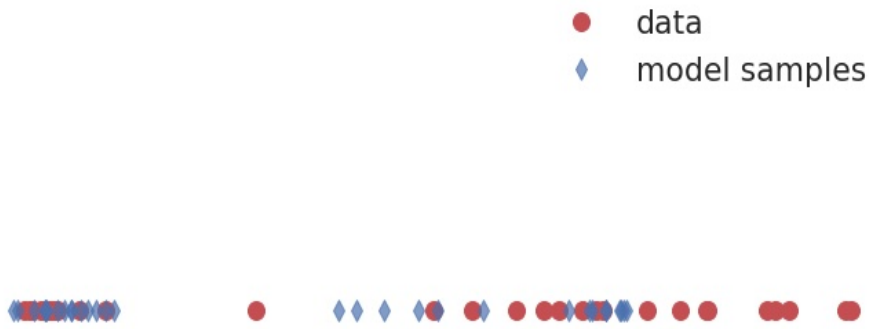


Image Credit: *Murphy 2023*

GAN setup

GAN setup

- **Goal:** We want to generate samples from $p_{true}(x)$.

GAN setup

- **Goal:** We want to generate samples from $p_{true}(x)$.
- Consider an objective :

$$L(p_{true}, q)$$

where q is a distribution we wish to learn parametrized by θ.

GAN setup

- **Goal:** We want to generate samples from $p_{true}(x)$.
- Consider an objective :

$$L(p_{true}, q)$$

  where q is a distribution we wish to learn parametrized by θ.
- Requirements:

GAN setup

- **Goal:** We want to generate samples from $p_{true}(x)$.
- Consider an objective :

$$L(p_{true}, q)$$

where q is a distribution we wish to learn parametrized by θ.

- Requirements:
  1. $$\underset{\theta}{\text{argmin}}\, L(p_{true}, q_\theta) = p_{true}$$

GAN setup

- **Goal:** We want to generate samples from $p_{true}(x)$.
- Consider an objective :

$$L(p_{true}, q)$$

where q is a distribution we wish to learn parametrized by θ.

- Requirements:
  1. $$\underset{\theta}{\operatorname{argmin}}\, L(p_{true}, q_\theta) = p_{true}$$
  2. Must be computable just using data samples from both data and model distributions.

GAN setup

- **Goal:** We want to generate samples from $p_{true}(x)$.
- Consider an objective :

$$L(p_{true}, q)$$

where q is a distribution we wish to learn parametrized by θ.

- Requirements:
  1. $$\underset{\theta}{\operatorname{argmin}}\, L(p_{true}, q_\theta) = p_{true}$$
  2. Must be computable just using data samples from both data and model distributions.
  3. Must be computationally cheap to evaluate.

GAN: Density Ratio

GAN: Density Ratio

- How do we design the comparison function L which satisfies the above requirements?

GAN: Density Ratio

- How do we design the comparison function L which satisfies the above requirements?
- One way : Compute Density Ratio :

$$r(x) = \frac{p_{true}(x)}{q_\theta(x)}$$

GAN: Density Ratio

- How do we design the comparison function L which satisfies the above requirements?
- One way : Compute Density Ratio :

$$r(x) = \frac{p_{true}(x)}{q_\theta(x)}$$

- How do we compute this with samples alone?

GAN: Density Ratio

- How do we design the comparison function L which satisfies the above requirements?
- One way : Compute Density Ratio :

$$r(x) = \frac{p_{true}(x)}{q_\theta(x)}$$

- How do we compute this with samples alone?
- To do this, consider a binary classification problem where samples from p_{data} have label $y = 1$ and and samples from q_θ are labelled as 0.

GAN: Density Ratio

- How do we design the comparison function L which satisfies the above requirements?
- One way : Compute Density Ratio :

$$r(x) = \frac{p_{true}(x)}{q_\theta(x)}$$

- How do we compute this with samples alone?
- To do this, consider a binary classification problem where samples from p_{data} have label $y = 1$ and and samples from q_θ are labelled as 0.
- **Claim:** Using just this binary classifier we can compute a distributional divergence using only samples.

GAN : Binary Classification

GAN : Binary Classification

- The density ratio $r(x)$ can be written in terms of the joint distribution $p(x, y)$ as:

$$\frac{p_{true}(x)}{q_\theta(x)} = \frac{p(x|y=1)}{p(x|y=0)}$$

GAN : Binary Classification

- The density ratio $r(x)$ can be written in terms of the joint distribution $p(x, y)$ as:

$$\frac{p_{true}(x)}{q_\theta(x)} = \frac{p(x|y=1)}{p(x|y=0)}$$

$$= \frac{p(y=1|x)p(x)}{p(y=1)} \Big/ \frac{p(y=0|x)p(x)}{p(y=0)}$$

GAN : Binary Classification

- The density ratio $r(x)$ can be written in terms of the joint distribution $p(x, y)$ as:

$$\frac{p_{true}(x)}{q_\theta(x)} = \frac{p(x|y=1)}{p(x|y=0)}$$

$$= \frac{p(y=1|x)p(x)}{p(y=1)} \Big/ \frac{p(y=0|x)p(x)}{p(y=0)}$$

$$\frac{p(y=1|x)}{p(y=0|x)} \frac{p(y=0)}{p(y=1)}$$

GAN : Binary Classification

- Let $p(y = 1) = \pi$ be the class prior.

$$\frac{p(y = 1|x)}{p(y = 0|x)} \frac{1 - \pi}{\pi}$$

GAN : Binary Classification

- Let $p(y = 1) = \pi$ be the class prior.

$$\frac{p(y = 1|x)}{p(y = 0|x)} \frac{1 - \pi}{\pi}$$

- Setting $\pi = 0.5$ gives us:

$$r(x) = \frac{p(y = 1|x)}{p(y = 0|x)}$$

GAN : Binary Classification

- Let $p(y = 1) = \pi$ be the class prior.

$$\frac{p(y = 1|x)}{p(y = 0|x)} \frac{1 - \pi}{\pi}$$

- Setting $\pi = 0.5$ gives us:

$$r(x) = \frac{p(y = 1|x)}{p(y = 0|x)}$$

- This can be estimated by fitting a binary classifier $D_\phi(x) = p(y = 1|x)$.

$$r(x) = \frac{D_\phi(x)}{1 - D_\phi(x)}$$

GAN: Discriminator

GAN: Discriminator

- The parameter ϕ can be estimated using maximum likelihood estimation as follows:

GAN: Discriminator

- The parameter ϕ can be estimated using maximum likelihood estimation as follows:

$$\underset{\phi}{\text{argmax}} \sum_{i=1}^{N} \log p(y_i|x_i)$$

GAN: Discriminator

- The parameter ϕ can be estimated using maximum likelihood estimation as follows:

$$\underset{\phi}{\text{argmax}} \sum_{i=1}^{N} \log p(y_i|x_i)$$

- Since $p(y_i|x_i) = D_\phi(x_i)$ is a binary classifier, this reduces to:

GAN: Discriminator

- The parameter ϕ can be estimated using maximum likelihood estimation as follows:

$$\underset{\phi}{\text{argmax}} \sum_{i=1}^{N} \log p(y_i|x_i)$$

- Since $p(y_i|x_i) = D_\phi(x_i)$ is a binary classifier, this reduces to:

$$\underset{\phi}{\text{argmax}}\, \mathbb{E}_{p(x|y)p(y)}[y \log D_\phi + (1-y)\log(1-D_\phi)]$$

GAN: Discriminator

- The parameter ϕ can be estimated using maximum likelihood estimation as follows:

$$\underset{\phi}{\text{argmax}} \sum_{i=1}^{N} \log p(y_i|x_i)$$

- Since $p(y_i|x_i) = D_\phi(x_i)$ is a binary classifier, this reduces to:

$$\underset{\phi}{\text{argmax}}\, \mathbb{E}_{p(x|y)p(y)}[y \log D_\phi + (1-y)\log(1-D_\phi)]$$

$$\underset{\phi}{\text{argmax}}\, \mathbb{E}_{p_{x|y=1}p_{(y=1)}}[\log D_\phi(x)] + \mathbb{E}_{p(x|y=1)p_{(y=0)}}[\log(1-D_\phi(x))]$$

GAN: Discriminator

- The parameter ϕ can be estimated using maximum likelihood estimation as follows:

$$\underset{\phi}{\text{argmax}} \sum_{i=1}^{N} \log p(y_i|x_i)$$

- Since $p(y_i|x_i) = D_\phi(x_i)$ is a binary classifier, this reduces to:

$$\underset{\phi}{\text{argmax}}\, \mathbb{E}_{p(x|y)p(y)}[y \log D_\phi + (1-y)\log(1-D_\phi)]$$

$$\underset{\phi}{\text{argmax}}\, \mathbb{E}_{p_{x|y=1}p_{(y=1)}}[\log D_\phi(x)] + \mathbb{E}_{p(x|y=1)p_{(y=0)}}[\log(1-D_\phi(x))]$$

$$\underset{\phi}{\text{argmax}}\, \frac{1}{2}\mathbb{E}_{p_{true}(x)}[\log D_\phi(x)] + \frac{1}{2}\mathbb{E}_{q_\theta(x)}[\log(1-D_\phi(x))]$$

GAN:Optimal Discriminator

GAN:Optimal Discriminator

- What is an optimal discriminator $D^*(x)$?

GAN:Optimal Discriminator

- What is an optimal discriminator $D^*(x)$?
- From the density ratio:

$$r(x) = \frac{p_{true}(x)}{q_\theta(x)} = \frac{D(x)}{1 - D(x)}$$

GAN:Optimal Discriminator

- What is an optimal discriminator $D^*(x)$?
- From the density ratio:

$$r(x) = \frac{p_{true}(x)}{q_\theta(x)} = \frac{D(x)}{1 - D(x)}$$

- Therefore, the optimally trained discriminator is given by:

$$D^*(x) = \frac{p_{true}(x)}{p_{true}(x) + q_\theta(x)}$$

GAN: Objective function

GAN: Objective function

- We showed that the objective for the discriminator is:

GAN: Objective function

- We showed that the objective for the discriminator is:

$$V(q_\theta, p_{true}) = \operatorname*{argmax}_\phi \frac{1}{2}\mathbb{E}_{p_{true}(x)}[\log D_\phi(x)] + \frac{1}{2}\mathbb{E}_{q_\theta(x)}[\log(1 - D_\phi(x))]$$

GAN: Objective function

- We showed that the objective for the discriminator is:

$$V(q_\theta, p_{true}) = \underset{\phi}{\text{argmax}} \frac{1}{2}\mathbb{E}_{p_{true}(x)}[\log D_\phi(x)] + \frac{1}{2}\mathbb{E}_{q_\theta(x)}[\log(1 - D_\phi(x))]$$

- If the discriminator is optimal we have:

$$V^*(q_\theta, p_{true}) = \frac{1}{2}\mathbb{E}_{p_{data}(x)}[\log D^*(x)] + \frac{1}{2}\mathbb{E}_{q_\theta(x)}[\log(1 - D^*(x))]$$

$$= \mathbb{E}_{p_{true}(x)}[\log \frac{p_{true}(x)}{p_{true}(x) + q_\theta(x)}] + \mathbb{E}_{q_\theta(x)}[\log \frac{q_\theta(x)}{p_{true}(x) + q_\theta(x)}]$$

$$= \mathbb{E}_{p_{true}(x)}[\log \frac{2p_{true}(x)}{p_{true}(x) + q_\theta(x)/2}] + \mathbb{E}_{q_\theta(x)}[\log \frac{2q_\theta(x)}{p_{true}(x) + q_\theta(x)/2}]$$

$$= \mathbb{E}_{p_{true}(x)}[\log \frac{p_{true}(x)}{p_{true}(x) + q_\theta(x)/2}] + \mathbb{E}_{q_\theta(x)}[\log \frac{q_\theta(x)}{p_{true}(x) + q_\theta(x)/2}] + \log 4$$

GAN: Objective function

$$= \mathbb{E}_{p_{true}(x)}[\log \frac{p_{true}(x)}{p_{true}(x) + q_\theta(x)/2}] + \mathbb{E}_{q_\theta(x)}[\log \frac{q_\theta(x)}{p_{true}(x) + q_\theta(x)/2}] + \log 4$$

$$= D_{KL}(p_{true} || \, p_{true}(x) + q_\theta(x)/2) + D_{KL}(q_\theta || \, p_{true}(x) + q_\theta(x)/2) + \log 4$$

$$= \log 4 + 2JSD(p_{true} || q_\theta)$$

- Where JSD is the Jensen Shannon Divergence defined as:

$$JSD(p_{true} || q_\theta) = \frac{1}{2} D_{KL}(p_{true} || \, p_{true}(x) + q_\theta(x)/2) + \frac{1}{2} D_{KL}(q_\theta || \, p_{true}(x) + q_\theta(x)/2) \quad (1)$$

GAN: Objective Function

GAN: Objective Function

- Hence, using just a binary classification we were able to compute a measure of the distance between the true and model distributions.

GAN: Objective Function

- Hence, using just a binary classification we were able to compute a measure of the distance between the true and model distributions.
- We would like to minimize the JSD between the two distributions.

$$\min_{\theta} JSD(p_{true}, q_\theta) = \min_{\theta} V^*(p_{true}, q_\theta) + \log 4$$
$$= \min_{\theta} \mathbb{E}_{p_{data}(x)}[\log D^*(x)] + \mathbb{E}_{q_\theta(x)}[\log(1 - D^*(x))]$$

- But we do not have access to the optimal discriminator, but only the parameterized discriminator $D_\phi(x)$.

$$\min_{\theta} \max_{\phi} \mathbb{E}_{p_{data}(x)}[\log D_\phi(x)] + \mathbb{E}_{q_\theta(x)}[\log(1 - D_\phi(x))]$$

Training Algorithm

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Generated Outputs

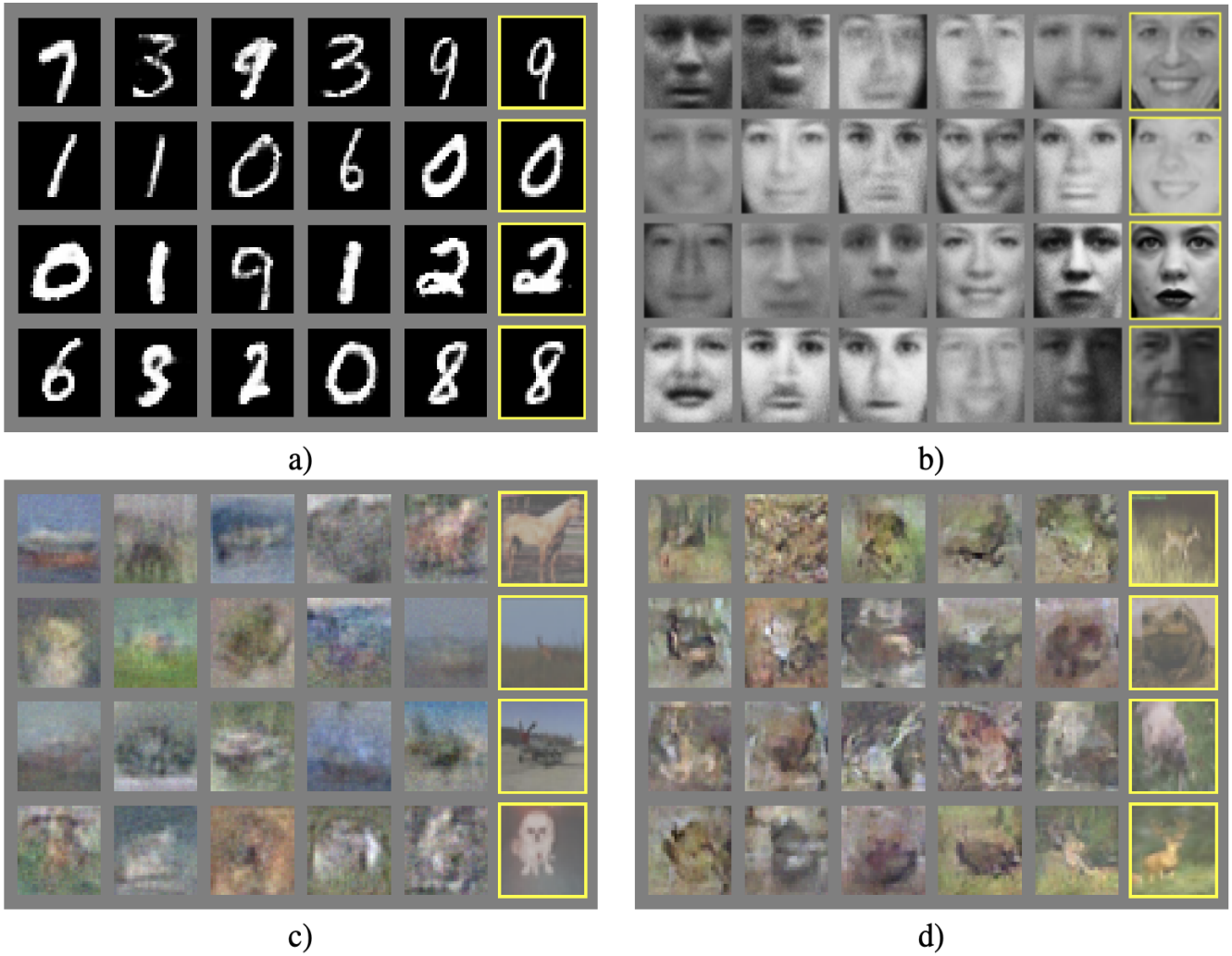


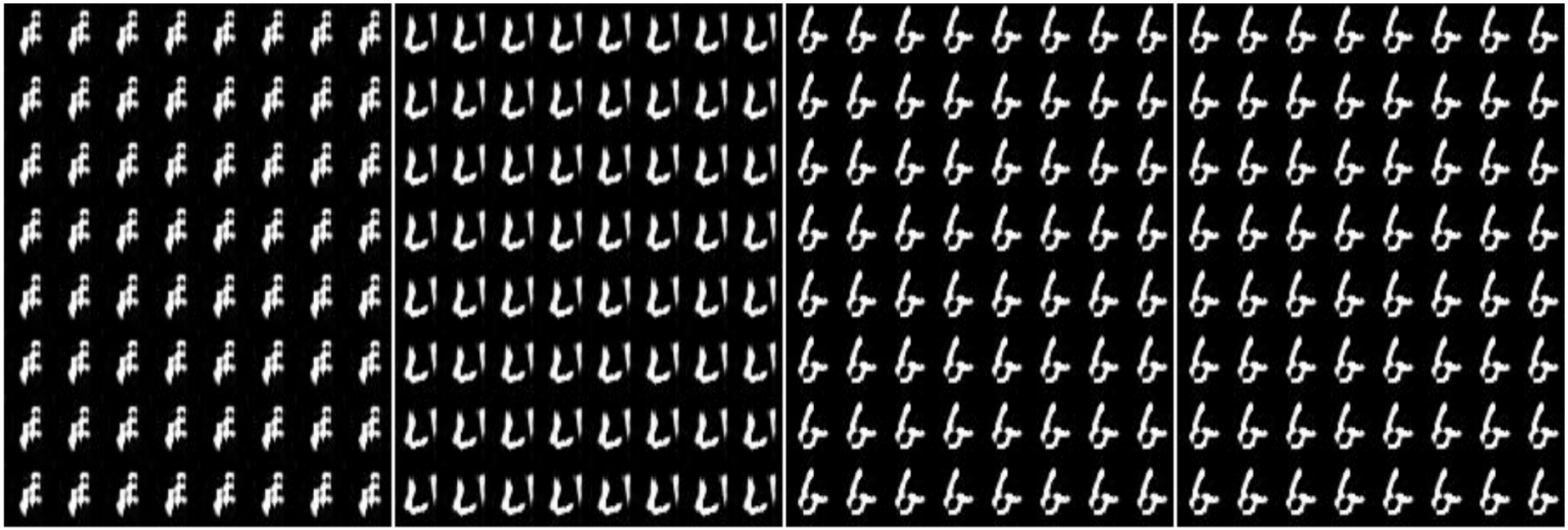
Image Credit: *Goodfellow et al. 2014*

Drawbacks

- **Mode Collapse**: The generator learns to generate only a few modes of the true distribution.
- **Vanishing Gradients**: The gradients of the discriminator vanish when the discriminator is optimal.

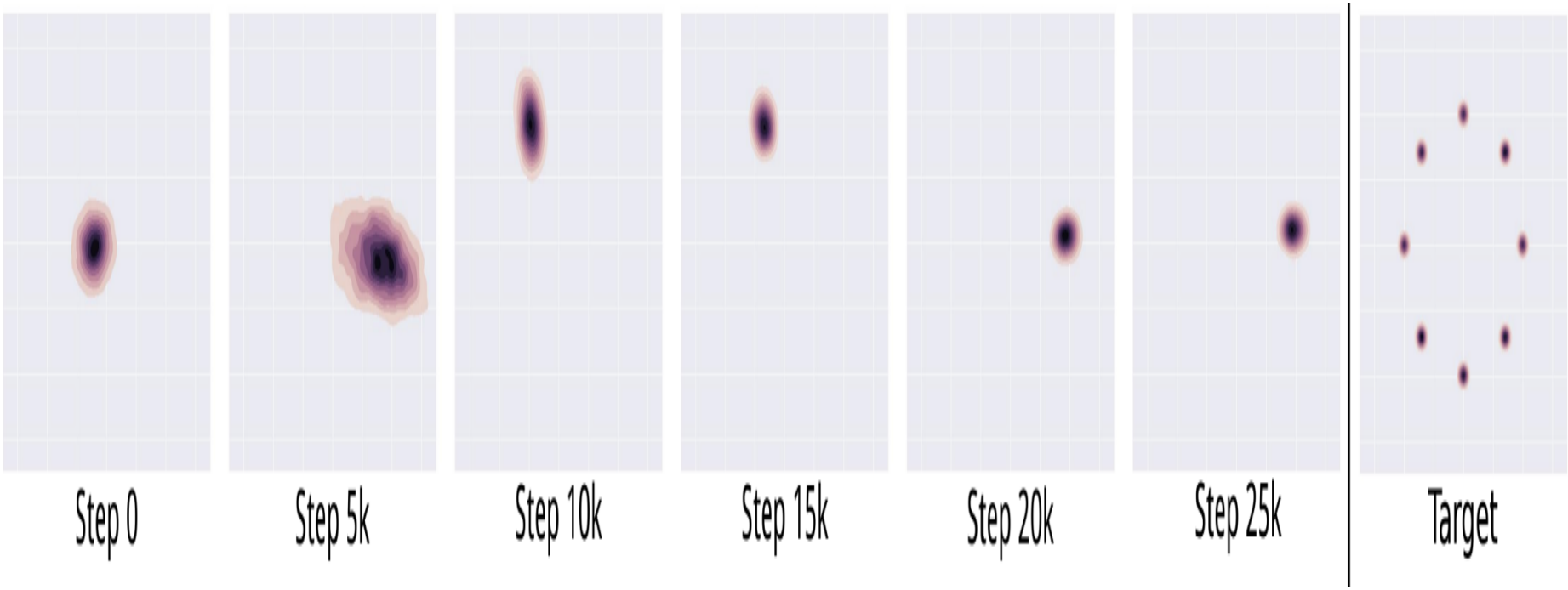
Mode Collapse

- **Mode Collapse**: The generator learns to generate only a few modes of the true distribution.



Mode Collapse

- Example of Mode Collapse in a GAN fitted on Gaussian training data of two modes.



Mode Collapse : Why

- **Why does mode collapse happen?**

Mode Collapse : Why

- **Why does mode collapse happen?**
- One explanation is because GAN minimized the JSD between the true and model distributions.

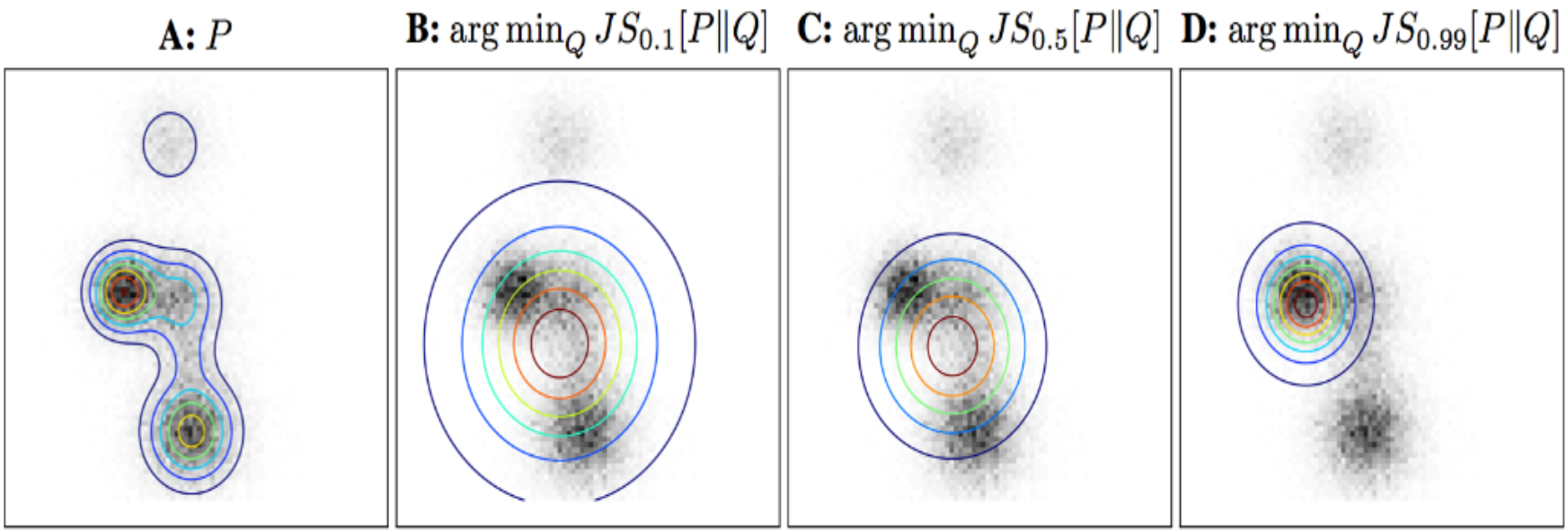


Image Credit: *Huszar et al. 2015*

Mode Collapse: Why

- **Why does mode collapse happen?**

Mode Collapse: Why

- **Why does mode collapse happen?**
- One explanation is because GAN minimized the JSD between the true and model distributions.

Mode Collapse: Why

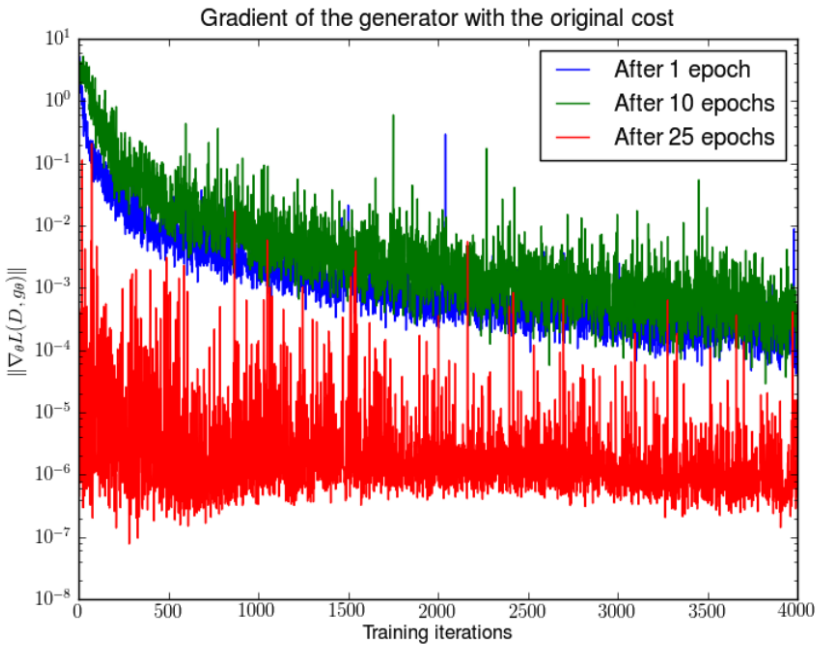
- **Why does mode collapse happen?**
- One explanation is because GAN minimized the JSD between the true and model distributions.
- But, this does not explain why GAN's that minimize the KL divergence also suffer from mode collapse.

Mode Collapse: Why

- **Why does mode collapse happen?**
- One explanation is because GAN minimized the JSD between the true and model distributions.
- But, this does not explain why GAN's that minimize the KL divergence also suffer from mode collapse.
- Understanding the reasoning behind mode collapse is still an open problem.

Vanishing Gradients

- **Vanishing Gradients**: The gradients of the discriminator vanish when the discriminator is optimal.



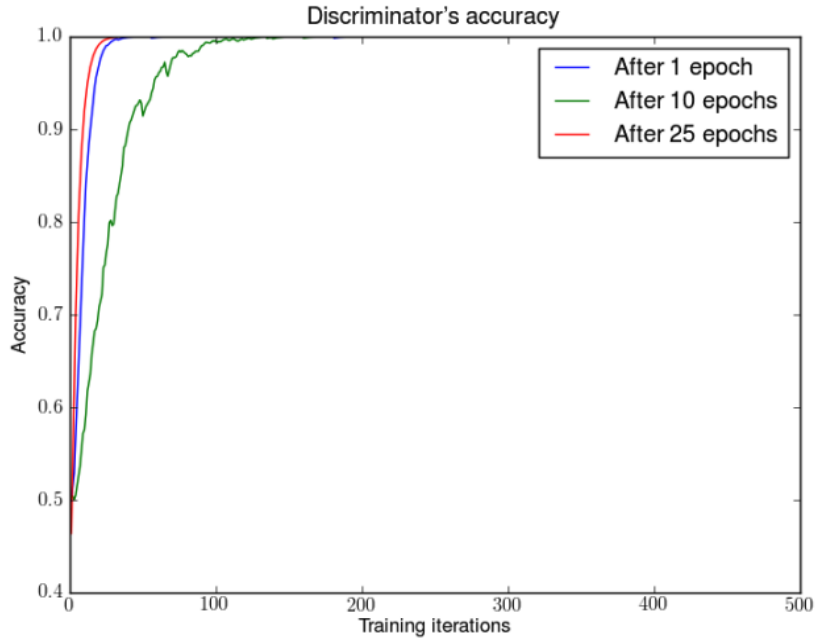


Image Credit: *Arjovsky et al. 2017*

Vanishing Gradients

Vanishing Gradients

- This is a problem since the generators cost function being an approximation of the JSD is dependant on the quality of the discriminator.

Vanishing Gradients

- This is a problem since the generators cost function being an approximation of the JSD is dependant on the quality of the discriminator.
- In order to mitigate this problem, the authors proposed a tweak to the generator cost function.

$$\min_{\theta} \mathbb{E}_{z \sim p(z)}[-\log D(G_\theta(z))]$$

- While this mitigates the problem of vanishing gradients, it causes unstable updates making it hard to train the generator.

GAN : Impact

Drawbacks

Future Work

Thank you!

Appendix